VALİDATİON

SERVER SİDE VALİDATİON

VALİDATİON OLUŞTURMA(DATA ANNOTATİONS)

Required Annotation

StringLenght Annotation

EmailAddress Annotation

CONTROLLER VALİDATİON KONTROLLÜ

ModelState

Validation Durumlarını Görme

Controller'da Validation Hata Mesajlarını Görme

View'de Validation Hata Mesajlarını Görme

Validation'lar genel bilgiler

VALİDATİON SORUMLULUĞUNU ENTİTY'DEN SOYUTLAMA

VALİDATİON'I BAŞKA BİR SINIFA YÜKLEME(MODELMETADATA TYPE)

FLUENT VALİDATİON KÜTÜPHANESİ

KÜTÜPHANENİN KURULUMU

VALİDATOR SINIFI OLUŞTURMA

VALİDATİON'LARI UYGULAMA

ÇALIŞMA MANTIĞI

CLİENT TABANLI VALİDATİON'LAR

SERVER'DAKİ VALİDATİON'LARI DİNAMİK OLARAK CLİENT TABANLI UYGULAMA

VALİDATİON'LARI VURGULAMA

Validation

Kullanıcıdan gelen verinin Server tarafında kontrol edilip doğrulanmasıdır.

Validation tanım olarak, bir değerin belirli koşullara ve amaca uygun olması, bu kontrollere göre verinin işleme tabi tutulması durumudur.

“Her Selamın aleyküm diyen veriyi alacak mıyız ?”

Genel olarak kullanıcıdan gelen veriler doğrulanarak işlem yapılır.

Validation ikiye ayrılık:

- Client Side Validation
- Server Side Validation

Önce Client Validation yapıp sonra Server Side Validation yapmak daha doğrudur. Çünkü Client tarafta doğrulanması gereken basit verilerin doğrulanması Server Side’daki maliyeti düşürebilir. Örneğin Tc kimlik numrasının 11 haneli olma durmunu Client tabanlı Validation ile yapmak daha mantıklıdır.

Sadece Client Side Validation yapmak yeterli değildir, garanti olması açısından hem Client Side hem de Server Side Validation yapılması gerekir. Çünkü kullanıcı Client side Validation’lara uygun fakat zararlı olabilecek verileri Server Side’a gönderebilir. Örneğin SQL Injection

Buradaki amaç hem kötü niyetli olmayan normal kullanıcılar için basit Validation’ları yaparken Server’a ekstra maaliyet yüklememek hem de kullanıcıya hızlı bir uyarı yapmak. Yani Mutualist bir ilişki var.

Validation’lar hayvan gibi İf-else’ler ile de yapılabilir ama bunu yapmak aşırı gereksiz ve mantıksız bir davranış.

Server Side Validation

Validation Oluşturma(Data Annotations)

Validation’lar Binding yapılan nesnenin Property’leri olarak yazılır.

Entity içerisinde Annonation isimli attribute’lar ile Validation’lar oluşturulabilir. Validation eklenmek istenen Property’nin üzerine gerekli Validation Attribute’ları yazılır.

Ayrıca Attribute’lar ile kullanılan ErrorMessage ile kullanıcıya ilgili Validation ile alakalı hata mesajı da döndürülebilir.

Required Annotation

Required, ilgili Property’nin boş gönderilemeyeceğini bildirir.

[Required(ErrorMessage ="Lütfen Name alanını boş bırakmayınız.")]

StringLenght Annotation

StringLenght, ilgili Property’nin en fazla kaç karakter olabilileceğini döndürür. İlk parametre olarak karakter sayısını ikinci parametre olarak da ErrorMessage alabilir.

[StringLength(100, ErrorMessage ="Lütfen name'i en fazla 100 karakter giriniz.")]

EmailAddress Annotation

EmailAdress, ilgili Property’nin email adres olup olmadığını doğrular.

[EmailAddress(ErrorMessage ="Lütfen doğru bir email adresi giriniz.")]

Controller Validation Kontrollü

ModelState

MVC'de bir type'ın data Annotations'larını kontrol eden ve geriye sonuç dönene Property.

Validation Durumlarını Görme

Controller'da validatiton yapmak için sadece ModelState'in IsValid Methodu kullanılır. Bu method'un True dönmesi gelen verinin bütün Validation'lardan başarılı bir şekilde çıktığını ve verinin kullanılabilir olduğunu gösterir.

Controller'da Validation Hata Mesajlarını Görme

Bu işlem Controller Tarafında hata mesajını çekmek için kullanılır View'de çekmek çok daha kolaydır.

ModelState içerisindeki Values'dan FirstOrDefault ile bir LinQ sorgusu yapılır. Bu sorgu, patlıyan ilk Validation'ı çekmek için kullanılır. Ardından patlıyan Validation'ın ilk hatasına girmek için Errors Collection'ını içerisindeki ilgili indexdeki Error mesajı alınır.

```
ViewBag.HataMesaj = ModelState.Values.FirstOrDefault(x =>
x.ValidationState ==
Microsoft.AspNetCore.Mvc.ModelBinding.ModelValidationState.Invalid).Errors[0].ErrorMessage;
```

View’de Validation Hata Mesajlarını Görme

View içerisinde Validation’ı olan input’lardan sonra Span’lar oluşturulur ve bu Span’lara Attribute olarak asp-validation-for verilir. Bu Attribute’a ise ilgili Validation’ının Property’si yazılır.

```
<form asp-action="CreateProduct" asp-controller="Product" method="post">
    <input type="text" asp-for="Name" name="name" value="" />
    <span asp-validation-for="Name"></span> <br />

    <input type="number" asp-for="Quantity" name="name" value="" />
    <span asp-validation-for="Quantity"></span> <br />

    <input type="email" asp-for="Email" name="name" value="" />
    <span asp-validation-for="Email"></span> <br />

    <button type="submit">Onayla</button>
</form>
```

Bu Validation mesajları için Controller tarafında ModelState’in False olması halinde View’e yeniden model gönderilir.

```
[HttpPost]
0 references
public IActionResult CreateProduct(Product model)
{
    if (!ModelState.IsValid) //Validation'lar dogrulanmazsa
    {
        //Kullanici bilgilendirmesi
        //Loglama vs.


        return View(model);
    }
    return View();
}
```

Validation’lar genel bilgiler

Validation’lar ile Bind edilen İnput nesneleri, Validation’lara uygun bir şekilde Client’a yansıtılır. Örneğin Max karakter sayısı 100 Validation’ını bir input nesnesine verilince İnput nesnesi otomatikmen maximum 100 karakter kabul edebilir hale gelir. Yani Validations’lar tarafına da yansır MVC sayesinde.

```
<input type="text" name="name" value data-val="true" data-val-length="Lütfen name'i en fazla 100 karakter giriniz." data-val-length-max="100"
val-required="Lütfen Name alanını boş bırakmayınız." id="Name" maxlength="100"> == $0
```

Client tabanlı Validation’ın güzelliği, bu Html kısımlarında yapılan Client tabanlı değişikler sayesinde kötü amaçlı kullanıcılar bu Validation’ları Bypass edebiliyor. Örneğin buradaki data-val-lenght-max değerini kullanıcı 250 yaparsa Server’a 250 karakter uzunluğunda veri gönderebilir. Fakat eğer Server Side bir Validation da varsa bu durum engelleniyor.

Sayfaya gelen tüm Validation’ları hatalarını tek bir yerde toplamak için, bir div oluşturup o div’e asp-validation-summary=”All” attribute verilir. Bu sayede sayfaya gelen tüm Validation hataları burada gösterir.

Validation Sorumluluğunu Entity’den soyutlama

Validation’ların Entity’lerde tutulmaması gerekir. İdeal olan ModelView’lerde tutulmasıdır.

Validation’ları Entity içerisinde tutumak, aynı zamanda SOLID prensiplere yani her yapının bir sorumluluğu olma durumuna aykırıdır. Entity sınıfı veri tabanını modelleyen yapı mı yoksa Validation yapan bir yapı mı burada bir ikilik olur.

Kısaca amaç Validaton’ların Entity’ler içerisinde tanımlanmamasıdır.

Bu durumun iki çözümü vardır, ilki olan ModelMetaData Type Validation sorumluluklarını farklı sınıflara yükler. İkinci seçenek ise Fluent Validaton kütüphanesidir.

Validation’ı Başka Bir Sınıfa Yükleme(ModelMetaData Type)

Models klasörü altında ModelMetadataTypes isimli bir klasör oluşturulur.

Ardından ilgili Entity ismi ile Metadata isimli bir class oluşturulur. Bu isimlendirme zorunlu değil raconsaldır.

Örneğin: ProductMetadata.cs

Bu sınıf içerisine Entity’de Validation uygulanacak olan Property’ler ve Annotation’ları yazılır. Entity içerisinde hiçbir Annotation kalmaz.

```
public class ProductMetadata
{
    [Required(ErrorMessage = "Lütfen Name alanını boş bırakmayınız.")]
    [StringLength(100, ErrorMessage = "Lütfen name'i en fazla 100 karakter giriniz.")]
    0 references
    public string Name { get; set; }



    [EmailAddress(ErrorMessage = "Lütfen doğru bir email adresi giriniz.")]
    0 references
    public string Email { get; set; }
}
```

Bu iki sınıfı ilişkilendirmek için, yani Metadata sınıfının hangi sınıfın Validation’larını taşıdığını bildirmek için ilgili Entity’e ModelMetadataType(Typeof(Metadata Sınıfı)) Attribute’u yazılır.

```
[ModelMetadataType(typeof(ProductMetadata))]
0 references
public class Product
{

    0 references
    public int Id { get; set; }
    0 references
    public string Name { get; set; }
    0 references
    public int Quantity { get; set; }
    0 references
    public string Email { get; set; }
}
```

Fluent Validation Kütüphanesi

Hem SOLID prensibleri sağlamak için hem de yönetilebilirdiği arttırabilmek için Fluent Validation kütüphanesi kullanılabilir.

En çok kullanılan Validation kütüphanelerinden biridir.

Kütüphanenin Kurulumu

Kütüphaneyi yüklemek için, Nuget üzerinden FluentValidation, FluentValidation.DependencyInjectionExtension indirmek yeterli.

Bu adımın ardından bu servisi projeye entegre etmek gerekir. Bunun için Program.cs içerisinde builder.Services.AddControllersWithViews(); satırının altına:

builder.Services.AddFluentValidationAutoValidation();

builder.Services.AddFluentValidationClientsideAdapters();

builder.Services.AddValidatorsFromAssemblyContaining<Program>();

Servislerini eklemek gerekir.

```
// Add services to the container.
builder.Services.AddControllersWithViews();
builder.Services.AddFluentValidationAutoValidation();
builder.Services.AddFluentValidationClientsideAdapters();

builder.Services.AddValidatorsFromAssemblyContaining<Program>();
```

AddValidatorsFromAssemblyContaining Validation’ları otomatik bulur.

Validator Sınıfı Oluşturma

Öncelikle Models altına bir Validators isimli bir klasör oluşturulur.

Bu klasörün altına ilgili Entity'lerin Validation sınıfları yazılır.

Örn: ProductValidator

Bu sınıfın hangi sınıfın Validator'ı olduğunu belirtmek için, AbstractValidator sınıfından kalıtılır. Abtract Validator Generic olarak Validation yapılacak olan sınıfı alır.

```
public class ProductValidatior : AbstractValidator<Product>
```

Kısaca sistem AbtractValidator sınıfından türeyen tüm sınıfların Validator olduğunu anlayacak.

Validation'ları Uygulama

Validator sınıfı oluşturduktan sonra Consturactor kullanarak Validation'lar yazılır.

RuleFor fonksiyonu kullanılır. RuleFor fonksiyonu parametre olarak uygulanacak olan Validation'ın hangi Property'ye yapılacağını alır.

Ardından hangi Validation'lar yapılacaksa "." İle Validation fonksiyonları çağırılır.

```
public ProductValidatior()
{
    RuleFor(x => x.Email).NotNull().WithMessage("Email boş olamaz!");
    RuleFor(x => x.Email).EmailAddress().WithMessage("Lütfen doğru bir email giriniz!");

    RuleFor(x => x.Name).NotNull().NotEmpty().WithMessage("Lütfen Prodcut Name'i Boş geçmeyiniz!");
    RuleFor(x => x.Name).NotNull().MaximumLength(100).WithMessage("Lütfen product Name'i maximum 100 karakter giriniz!");
}
```

İlgili Validation'ın onaylanmaması durumunda WithMessage ile mesaj yazdırırlır.

Validation Fonksiyonları ard arda kullanılabilir.

Burada "Requared" yok onun yerine NotNull veya NutEmpty var.

Bazı Validation Fonksiyonları:

- NotNull
- NotEmpty
- EmailAdress
- MaximumLenght – Parametre olarak uzunluğu alır
- LessThen – Parametre olarak uzunluğu alır

Validation'ları tanımladıktan sonra ekstra birşey yapmaya gerek yoktur. Çünkü Program.cs içerisinde "`AddValidatorsFromAssemblyContaining<Program>()`" fonksiyonu sayesinde, kütüphane otomatik bir şekilde AbstractValidator sınıfından türeyen sınıfları Validator olarak tanımlar ve bu AbstractValidator'a Generic olarak verilen sınıflar üzerinden ilgili Validator sınıflarını eşleştirir.

Çalışma Mantığı

Controller tarafında Port edilen Action'a gelen model(Product örn.) ilgili Validation'lar kontrol edilerek gelir. Bu kontrol sırasında ModelState doldurulur. Eğer Validation'lar sağlanmıyorsa ModelState içerisindeki ilgili Error mesajları doldurulacak. View kısmında Bind edilen ilgili yerlere mesajlar gösterilecektir.

Sayfa Render edilirken ilgili Validation'lar input nesnelerine yerleştirilir.

Art arda tanımlanan Validation fonksiyonlarında her Validation fonksiyonundan sonra ilgili Hata mesajını yazdırıp ardından diğer Validation fonksiyonunu yazmak gerekir.

```
RuleFor(x => x.Name).NotNull().WithMessage("Lütfen Prodcut Name'i Null geçmeyiniz!").NotEmpty().WithMessage("Lütfen Prodcut Name'i Boş geçmeyiniz!");
```

Eğer bir Validation ile alaklı herhangi bir mesaj girilmezse default mesaj yazdılır.

HTTP Request

↓

Model Binding (JSON → C# nesnesi)

↓

ModelState oluşur

↓

FluentValidation çalışır (EĞER model başarılı bind olduysa)

↓

Controller Action

Client Tabanlı Validation’lar

İdeal seneryoda hem Client üzerinde hem de Server üzerinde Validation yapmak gerekir.

Client-side validation, basit ve hızlı doğrulamaları sunucuya göndermeden yakalayarak gereksiz server yükünü azaltmak ve kullanıcıya anlık geri bildirim vererek kullanıcı deneyimini artırmak için kullanılır.**Server-side Server-side validation** ise client tarafından değiştirilebilen veriye güvenmeyerek, bypass edilmiş doğrulamaları yakalamak, veri bütünlüğünü korumak, iş kurallarını uygulamak ve sistem güvenliğini sağlamak için zorunlu olarak yapılır.

Her türlü Validation için yani Fluent Validation, Data Annotations, metadata fark etmeksizin bu Validation’ları Client’da yeniden yazmaya gerek duymadan direkt Server’dan Validation’ları çekerek “Client Tabanlı Validation” yapma.

Server’daki Validation’ları Dinamik Olarak Client Tabanlı Uygulama

Backend’deki Validation’ları Frontend’de(Client’da) kullanmak için Javascript, Jquery gibi farklı yollar da vardır. Ama daha yönetilebilir ve daha düzgün bir yapı olması için hali hazırda yazılmış olan Validation’ları Backend’den çekip kullanmak daha doğru bir yaklaşım. Aynı zamanda Validation’lar aynı yerden çekildiği için yönetilebiliriği çok daha yüksektir.

Bu iş için 3 adet kütüphane gerekmektedir.

Frontend Kütüphaneleri:

- Jquery
- Jquery-validate
- Jquery-validation-unobtrysive – Microsoft geliştirdi

Validation’ları Vurgulama

Validation’ları uygulamak için Validation yapılacak olan View içerisine ilgili Jquery kütüphanelerinin min dosyalarını Script olarak vermek gerekiyor.

```
<script src="~/jquery/jquery.min.js"></script>
<a href="~/jquery/jquery.min.map">~/jquery/jquery.min.map</a>
<script src="~/jquery-validate/jquery.validate.min.js"></script>
<script src="~/jquery-validation-unobtrusive/jquery.validate.unobtrusive.min.js"></script>
```

Bu işlemlerden sonra herhangi bir Validation onaylanmazsa istek hiç Server’a gitmez Client’de kullanıcıya hata mesajı verilir.

Validation’lar input nesnelerine otomatik olarak verilir.

Ayrıca bu Validation’lar dinamik olarak çalışır yani kullanıcı doğru email girmediği zaman sadece form submit olunca değil her anda uyarı mesajı gösterilir kullanıcıya.